

Binary Classification and Clustering Analysis of Handwritten Digits 1 and 7 from the MNIST Dataset

Md Soad Solaiman

Abstract

This paper investigates binary classification and unsupervised clustering on a subset of the MNIST handwritten digit dataset restricted to digits 1 and 7. Using 100 labelled training images and 1,000 labelled test images, exploratory analysis characterises the class distributions, pixel-level statistics, and two-dimensional principal component structure of the data. Two classifiers, Logistic Regression and a Radial Basis Function (RBF) kernel Support Vector Machine, are trained and compared using stratified five-fold cross-validation and grid-search hyperparameter tuning. Logistic Regression, tuned to $C = 0.1$ with L2 regularisation, achieves a mean cross-validation F1-score of 1.000, outperforming the best RBF SVM configuration ($C = 10$, $\gamma = 0.001$), which reaches 0.970. These results indicate that the two digit classes are close to linearly separable and that additional model complexity is unnecessary for this task. In a separate unsupervised analysis, K-Means clustering is applied to the 500 test images of digit 7 to characterise handwriting-style variation. The elbow method and silhouette analysis jointly identify $k = 9$ as a suitable number of clusters, revealing distinct stroke styles, sizes and orientations. The resulting cluster sizes are reasonably balanced, suggesting that K-Means recovers meaningful handwriting-style groupings without label supervision.

Keywords: MNIST; Handwritten Digit Recognition; Logistic Regression; Support Vector Machine; Principal Component Analysis; K-Means Clustering; Cross-Validation

1 Introduction

The Modified National Institute of Standards and Technology (MNIST) database is one of the most widely used benchmarks in machine learning research, constructed by re-mixing samples from NIST Special Databases 1 and 3 (LeCun et al., 1998). Its accessibility and well-defined structure have made it a standard testbed for evaluating classification algorithms, from early linear models to modern deep networks.

This paper focuses on a reduced binary subset of MNIST containing only the digits 1 and 7, comprising 100 labelled training images and 1,000 labelled test images, each a 28×28 grayscale array. Restricting the problem to two visually distinct digit classes allows two related but separate questions to be examined in a lower-dimensional, but still realistic, setting:

1. **Classification.** How well can supervised classifiers distinguish between digits 1 and 7 given only a small training set, and how does a linear classifier compare to a non-linear kernel-based classifier once both are properly tuned?
2. **Clustering.** Independent of digit labels, what within-class structure exists among instances of a single digit? Specifically, how many distinct handwriting styles can be recovered from unlabelled examples of digit 7 using unsupervised clustering?

To answer the first question, Logistic Regression and a Radial Basis Function (RBF) kernel Support Vector Machine are trained, evaluated with stratified k -fold cross-validation, and tuned via grid search. To answer the second, K-Means clustering is applied to the digit-7 test images, with the number of clusters selected using the elbow method and silhouette analysis.

The remainder of this paper is organised as follows. Section 2 reviews related work on MNIST classification, dimensionality reduction, and clustering. Section 3 describes the dataset and presents exploratory analysis. Section 4 details the classification methodology, including model selection, evaluation metrics, and hyperparameter search strategy. Section 5 presents and discusses the classification results. Section 6 reports the unsupervised clustering analysis of digit 7. Section 7 concludes the paper and outlines directions for future work.

2 Related Work

The MNIST dataset was introduced by LeCun et al. (1998) as part of early work on gradient-based learning for document recognition, and it remains a common benchmark for evaluating new classification methods on image data. Principal Component Analysis (PCA), used here for exploratory visualisation, is reviewed extensively by Jolliffe and Cadima (2016), who describe its role in reducing high-dimensional feature spaces to a small number of directions of maximum variance while retaining interpretability.

For classification, Logistic Regression has repeatedly been shown to provide a strong, interpretable baseline for image-based tasks, particularly when training data is limited, as demonstrated by Karuppasamy et al. (2024) in the context of histopathology image classification. RBF kernel Support Vector Machines offer a non-linear alternative capable of modelling more complex decision boundaries and have likewise been applied successfully to image classification problems (Byun et al., 2024). Robust model selection for both approaches depends on an appropriate cross-validation strategy; Vasan and Surendiran (2024) compare leave-one-out, k -fold, and repeated k -fold cross-validation and highlight the importance of stratification when class balance must be preserved across folds.

On the unsupervised side, K-Means remains one of the most widely used clustering algorithms due to its simplicity and scalability, partitioning data into k clusters by minimising the within-cluster sum of squared distances to each centroid (Govender and Sivakumar, 2020). All classification and clustering procedures in this study were implemented using the scikit-learn library (Pedregosa et al., 2011), following the practical machine-learning workflow described by Géron (2022).

3 Dataset and Exploratory Analysis

3.1 Data Inspection

This study uses a binary subset of the MNIST handwritten digit database containing only samples of digits 1 and 7. The subset provides four arrays, summarised in Table 1.

Table 1: Dataset component shapes

Dataset Component	Shape	Description
Training Images	(100, 28, 28)	100 grayscale images, 28×28 pixels
Training Labels	(100)	Integer labels: 1 or 7
Test Images	(1000, 28, 28)	1,000 grayscale images, 28×28 pixels
Test Labels	(1000)	Integer labels: 1 or 7

3.2 Class Distribution

The training set consists of 50 samples of digit 1 and 50 samples of digit 7, producing a perfectly balanced binary classification problem (Table 2). This balance is beneficial for two reasons: no class is under-represented, which reduces the risk of a biased model, and no oversampling or class-weighting is required as a preprocessing step.

Table 2: Class distribution in the training set

Digit	Number of Training Samples
1	50
7	50
Total	100

3.3 Pixel Statistics

To characterise the pixel-level properties of each digit class, three statistics were computed across all training images: the average pixel intensity, the pixel variance, and the sparsity ratio (the proportion of near-zero, background pixels). Table 3 reports the results.

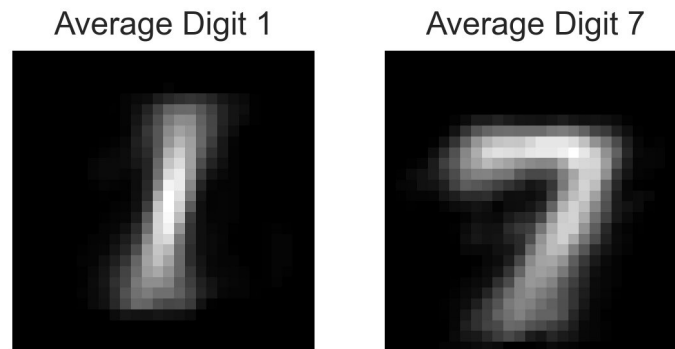
Table 3: Pixel statistics per digit class

Digit	Avg Pixel Intensity	Pixel Variance	Sparsity Ratio
1	20.18	4064.65	0.89
7	29.53	5606.38	0.83

Digit 7 has a higher average intensity and variance than digit 1, consistent with it occupying a visibly larger and more varied stroke area, while digit 1 is sparser, reflecting its narrow, concentrated form.

3.4 Average Digit Images

The average image for each class was computed by taking the mean pixel value, position-wise, across all training samples of that class. This produces a representative visual template for each digit as learned directly from the data.

**Figure 1:** Average pixel images for digit 1 (left) and digit 7 (right).

The average images in Figure 1 confirm the structural differences between the two digits. Digit 1 appears as a narrow stroke concentrated near the centre of the image, while digit 7 displays a clear horizontal crossbar at the top and a diagonal downstroke.

3.5 PCA Projection

The scaled training data was transformed from a 784-dimensional feature space into two dimensions via Principal Component Analysis (PCA). PCA finds orthogonal directions of maximum

variance in the data and is a useful tool for visualising high-dimensional feature spaces efficiently (Jolliffe and Cadima, 2016).

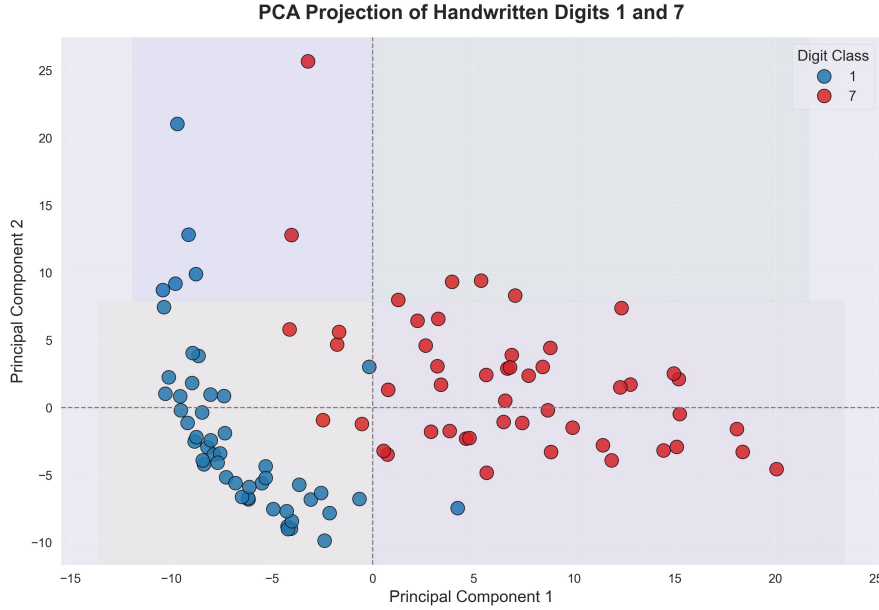


Figure 2: PCA projection of the training data, coloured by digit class.

The PCA scatterplot in Figure 2 shows a strong degree of linear separation between digit 1 (blue) and digit 7 (red), suggesting that a linear classifier can perform well on this task and motivating the model choice discussed in Section 4.

3.6 Data Preprocessing

Table 4 summarises the preprocessing pipeline applied before modelling.

Table 4: Preprocessing steps and justifications

Step	Operation
1. Flattening	Reshape $(N, 28, 28) \rightarrow (N, 784)$
2. Normalisation	Divide pixel values by 255.0 $\rightarrow [0, 1]$
3. Standardisation	StandardScaler : zero mean, unit variance

To prevent data leakage, `fit_transform` was applied only to the training set, and the resulting scaling parameters were then used to `transform` the test set. Test-set statistics were therefore never included in the calculation of the scaling parameters, preserving the integrity of all downstream evaluations (Géron, 2022). The scikit-learn library was used to implement all machine-learning procedures reported in this paper (Pedregosa et al., 2011).

4 Classification Methodology

4.1 Model Selection

Logistic Regression was selected as the primary classification algorithm because this is fundamentally a binary classification problem, and logistic regression has been shown to provide a strong baseline classifier for image-based tasks, particularly when training data is limited (Karuppasamy et al., 2024). This choice was supported by four data-informed considerations:

- **Linear separability observed in PCA.** As shown in Figure 2, digits 1 and 7 are close to linearly separable in the principal-component space.
- **Small training set (100 samples).** With such a small sample, complex non-linear models risk overfitting the training data. Logistic Regression is a comparatively low-variance model with a built-in regularisation parameter (C) suited to small sample sizes.
- **High-dimensional input (784 features).** Despite the high nominal dimensionality, MNIST images are sparse; Logistic Regression handles sparse, high-dimensional data efficiently, particularly with L1 or L2 regularisation.
- **Interpretability and computational efficiency.** Logistic Regression produces a linear decision boundary with interpretable weights, and training is fast with the `liblinear` solver, making iterative hyperparameter tuning via cross-validation feasible.

A second classifier, a Support Vector Machine with a Radial Basis Function (RBF) kernel, was selected to provide a deliberate contrast with Logistic Regression along several dimensions, summarised in Table 5.

Table 5: Comparison of model characteristics

Dimension	Logistic Regression	RBF Kernel SVM
Decision boundary type	Linear	Non-linear
Feature space	Original 784-dim. space	Implicitly infinite-dim. via RBF kernel
Optimisation objective	Minimises log-loss (cross-entropy)	Maximises margin between support vectors
Key hyperparameters	C (regularisation), penalty type	C (regularisation), γ (kernel width)
Probabilistic output	Yes (direct probability estimates)	Not native
Sensitivity to feature scale	Moderate	High

4.2 Evaluation Metrics

Multiple metrics were used to provide a comprehensive evaluation of classifier performance, listed in Table 6.

Table 6: Evaluation metrics and justifications

Metric	Definition	Justification for Use
Accuracy	Proportion of all correctly classified samples	Valid because the dataset is perfectly balanced (50/50)
Precision	$TP/(TP + FP)$ for the positive class (digit 7)	Measures how many predicted 7s were truly 7s
Recall	$TP/(TP + FN)$ for the positive class (digit 7)	Measures how many actual 7s were correctly identified
F1-Score	Harmonic mean of precision and recall	Provides a single balanced measure; used as the primary CV optimisation metric
CV F1-Macro	Average F1 computed equally across both classes	Appropriate for model selection, ensuring balanced performance across both digit classes

F1-macro was used as the primary scoring metric during cross-validation because it treats both classes equally.

4.3 Cross-Validation Strategy

Throughout this study, 5-fold Stratified K -Fold cross-validation (`StratifiedKFold`, `n_splits=5`, `shuffle=True`, `random_state=42`) was used for both model evaluation and hyperparameter tuning. Stratified K -Fold is recommended for classification problems because it guarantees that each fold preserves the same class distribution as the original dataset (Vasan and Surendiran, 2024). This property is particularly important given the small size of this dataset. To ensure repeatability, the Logistic Regression model was instantiated with `max_iter=1000` and `random_state=42`.

4.4 Hyperparameter Search Spaces

Two hyperparameters were tuned for Logistic Regression via grid search:

- **C (regularisation strength):** values tested $\{0.001, 0.01, 0.1, 1, 10, 100\}$. Lower C applies stronger regularisation; higher C allows more flexibility.
- **Penalty type:** L1 (lasso) and L2 (ridge). L1 produces sparse weight vectors, while L2 distributes weight more evenly. The `liblinear` solver was used, as it supports both penalty types.

Two hyperparameters were tuned for the RBF SVM via grid search:

- **C (regularisation strength):** values tested $\{0.1, 1, 10, 100\}$. Controls the trade-off between maximising the margin and minimising training error.
- **γ (kernel width):** values tested $\{0.001, 0.01, 0.1, 1\}$. Controls the influence radius of each support vector: a small γ gives broad influence, and a large γ gives narrow influence.

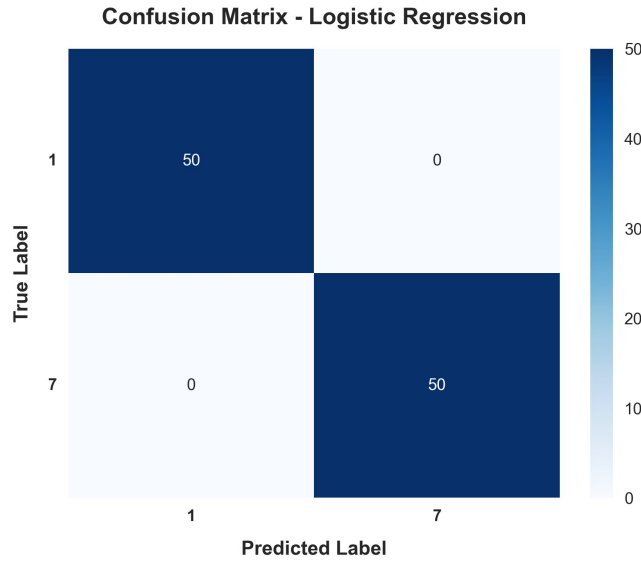
5 Classification Results and Discussion

5.1 Logistic Regression: Training and Cross-Validation Performance

Table 7 reports the training-set and cross-validation performance of Logistic Regression prior to hyperparameter tuning.

Table 7: Logistic Regression training and cross-validation results

Model	Train Acc	Train Prec	Train Recall	Train F1	Mean CV F1	CV Std
Logistic Regression	1.000	1.000	1.000	1.000	1.000	0.000

**Figure 3:** Confusion matrix for Logistic Regression on the training set.

The Logistic Regression classifier performed perfectly on the training set, scoring 1.000 across all metrics with no misclassifications for either class (Figure 3). The same trend held under 5-fold cross-validation, which produced a consistent mean F1-macro of 1.000 with a standard deviation of 0.000. Perfect training performance can be a symptom of overfitting; however, the consistency of the result across all five cross-validation folds strongly suggests that the model is genuinely generalising rather than memorising the training data. This is consistent with the high linear separability observed in the PCA projection (Figure 2), which makes Logistic Regression a well-suited model for this dataset.

5.2 Hyperparameter Tuning Results

Table 8 reports the best Logistic Regression configuration identified by grid search.

Table 8: Best Logistic Regression hyperparameters from GridSearchCV

Metric	Value
Best C	0.1
Best Penalty	L2
Best Mean CV F1-Score	1.0

For the RBF SVM, Figure 4 presents the full grid of mean cross-validation F1-scores across the C - γ search space, and Table 9 reports the best configuration.

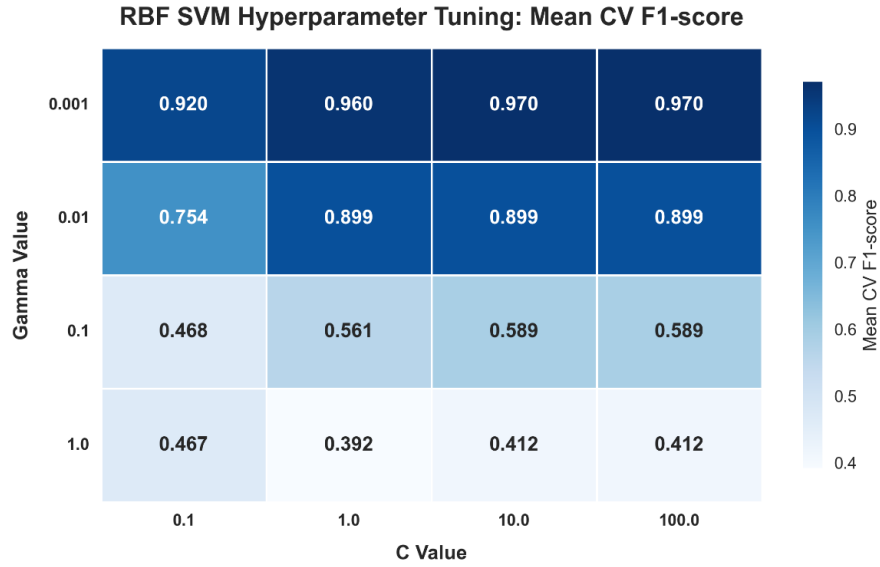


Figure 4: RBF SVM hyperparameter heatmap of mean CV F1-score (C vs. γ).

Table 9: Best RBF SVM hyperparameters from GridSearchCV

Best C	Best γ	Best Mean CV F1
10	0.001	0.970

5.3 RBF SVM Training Performance

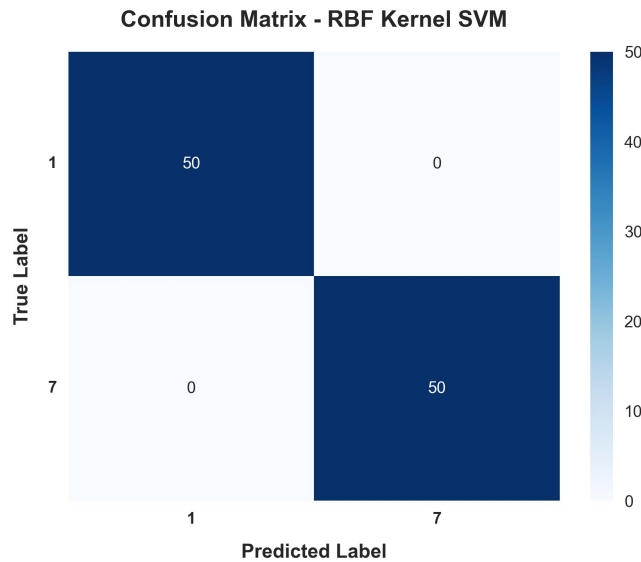


Figure 5: Confusion matrix for the RBF Kernel SVM on the training set.

As shown in Figure 5, both classifiers achieve perfect classification on the training set (50/50 correct for each class, zero misclassifications). This is consistent with both models having sufficient capacity to fit a well-separated binary problem with only 100 samples; it is the cross-validation results, reported above, that differentiate them.

5.4 Final Model Comparison and Discussion

Table 10 summarises the final comparison between the two tuned classifiers.

Table 10: Final model comparison after hyperparameter tuning

Model	Best Parameters	Best Mean CV F1	Training F1
Logistic Regression	$C = 0.1$, penalty = L2	1.000	1.000
RBF Kernel SVM	$C = 10$, $\gamma = 0.001$	0.970	1.000

Logistic Regression outperforms the RBF SVM in cross-validation (1.000 versus 0.970), indicating that a linear model is sufficient, and likely preferable, for this dataset. The RBF SVM appears to overfit the training set slightly relative to its cross-validated performance: the non-linearity of the kernel introduces additional model complexity that is not required for a problem that is nearly linearly separable.

The key difference between the two models is the nature of their decision boundary. Logistic Regression learns a flat hyperplane directly in the 784-dimensional input space, whereas the RBF SVM implicitly maps the data into a higher-dimensional space and learns a margin-maximising boundary within it. For this particular binary digit problem, the additional complexity of the RBF mapping makes generalisation somewhat more difficult rather than easier. Logistic Regression, with $C = 0.1$ and an L2 penalty, is therefore identified as the best model for this task: it is simple, easy to interpret, and achieves the strongest cross-validated performance.

6 Clustering Analysis of Handwritten Digit 7

6.1 Data Preparation

For the clustering analysis, only digit-7 images from the test set were used. This subset was prepared by selecting all test images with label 7, reshaping each 28×28 image into a flat 784-dimensional vector, and normalising pixel values to the $[0, 1]$ range. The resulting shapes are shown in Table 11.

Table 11: Digit 7 test data shapes

Data Component	Shape
Digit 7 Test Images	(500, 28, 28)
Digit 7 Feature Vectors (normalised)	(500, 784)

6.2 Baseline K-Means with 100 Clusters

As an initial baseline, K-Means clustering was applied with $k = 100$ clusters. K-Means is an unsupervised algorithm that partitions data into k clusters by minimising the within-cluster sum of squared distances to each cluster centroid (Govender and Sivakumar, 2020). The baseline was configured with `n_clusters=100`, `random_state=42` for reproducibility, and `n_init=10` to run the algorithm from ten different centroid initialisations and retain the best result.

6.3 Determining the Optimal Number of Clusters

Two standard methods were applied in combination to identify a more interpretable number of clusters than the $k = 100$ baseline.

6.3.1 Elbow Method (Inertia)

The inertia (the sum of squared distances from each point to its assigned cluster centroid) was computed for $k = 2$ to $k = 99$. Inertia always decreases as k increases. The elbow point, where the rate of decrease slows sharply, is therefore taken as an efficient choice for the number of clusters (Govender and Sivakumar, 2020).

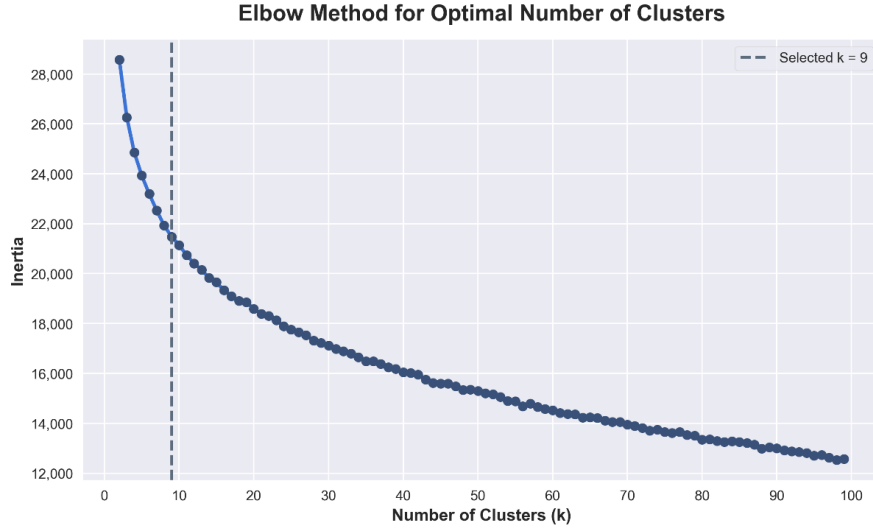


Figure 6: Elbow method: inertia versus number of clusters ($k = 2$ to 99).

As shown in Figure 6, inertia decreases sharply between $k = 2$ and approximately $k = 9$, after which the rate of decrease slows markedly. This suggests that increasing k beyond approximately 9 yields only a small further reduction in within-cluster variance.

6.3.2 Silhouette Analysis

The silhouette score measures how similar a point is to its own cluster compared with other clusters; higher values indicate more clearly separated and well-defined clusters.

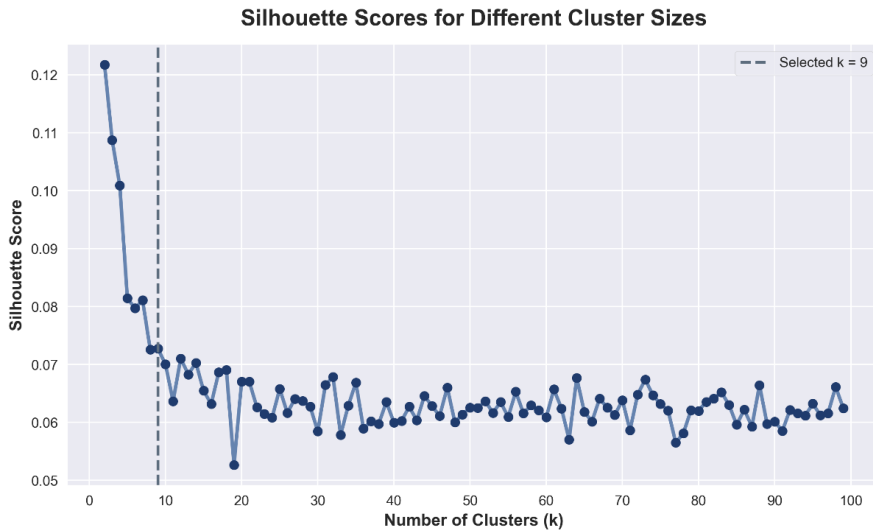


Figure 7: Silhouette scores for $k = 2$ to 99.

As shown in Figure 7, the silhouette score peaks at $k = 2$ (approximately 0.122) and decreases as k increases, converging to approximately 0.06 for $k > 20$. Although $k = 2$ achieves the

highest silhouette score, it is too coarse to capture meaningful variability among handwritten 7s. Notably, $k = 9$ corresponds to a local maximum in the silhouette curve, which aligns well with the elbow observed in Figure 6.

6.4 Final Cluster Selection

Both the elbow method and silhouette analysis point to $k = 9$ as a suitable number of clusters, for four reasons:

- **Elbow method:** at $k = 9$, the rate of decrease in inertia slows markedly, indicating that additional clusters beyond this point provide diminishing returns.
- **Silhouette score:** at $k = 9$, the silhouette score sits at a locally maximal and comparatively stable level, indicating reasonably well-separated clusters.
- **Interpretability:** nine clusters is a manageable number, with each cluster plausibly representing a distinct handwriting style. This is detailed enough to be informative without being unwieldy.
- **Practical constraint:** the task requires $k < 100$; $k = 9$ comfortably satisfies this while still producing meaningful, distinct clusters.

6.5 Cluster Centres and Interpretation

Each cluster centre is the mean of all images assigned to that cluster, representing the “average” appearance of digit 7 within it.

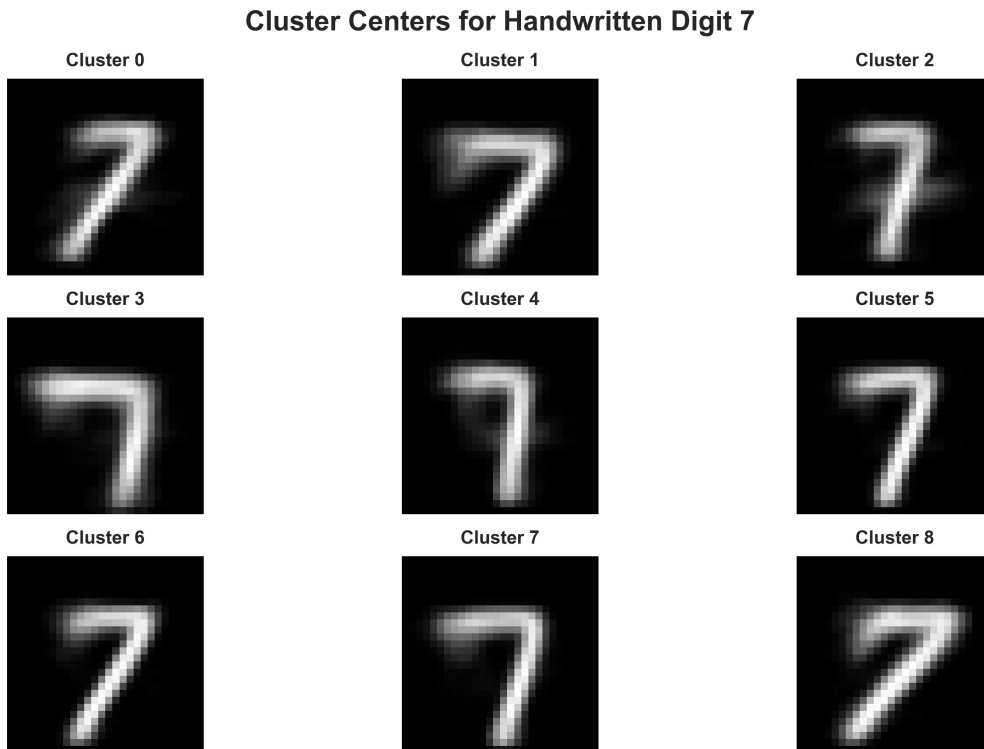


Figure 8: K-Means cluster centres ($k = 9$) for the digit 7 test images.

The cluster centres in Figure 8 reveal several distinct ways in which digit 7 was written:

- **Clusters 0, 1, 2, and 5:** clear, typical 7s with a straight top line and a diagonal stroke. This is the most common handwriting style.
- **Cluster 3:** an enlarged 7, shifted slightly to the left, suggesting a subset of writers who use larger strokes.

- **Cluster 4:** a smaller, more angular 7 with a more solid vertical stroke, visually closer to a “1 with a small top line”.
- **Clusters 6 and 7:** slightly tilted 7s, differing in writing angle.
- **Cluster 8:** a smaller 7 positioned lower and to the right within the image.

6.6 Cluster Size Distribution

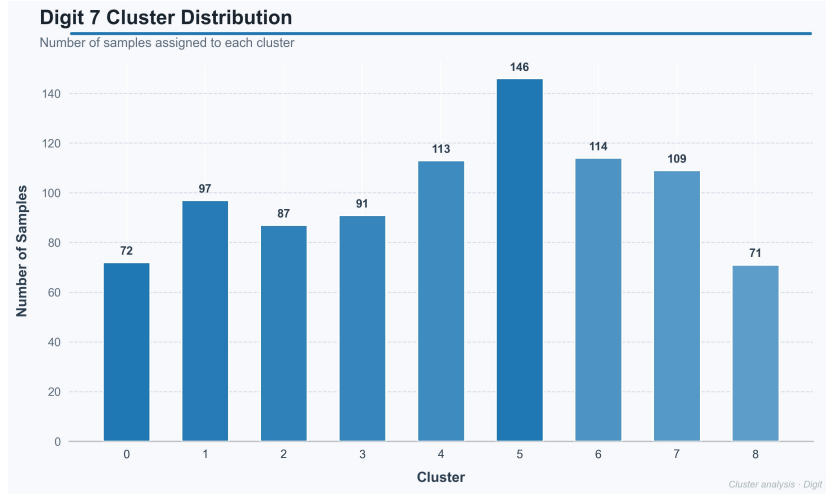


Figure 9: Distribution of samples per cluster for digit 7 ($k = 9$).

As shown in Figure 9, the most common handwriting style corresponds to cluster 5, with 146 images (18.5% of the digit-7 test set). By contrast, clusters 0 and 8 are the smallest, containing 71–72 images each (approximately 9%), indicating less common but still distinct handwriting styles. Overall, cluster sizes are reasonably balanced, reflecting a variety of handwriting styles for digit 7 and indicating that K-Means successfully discovered meaningful structure in the data.

6.7 Summary

The clustering analysis successfully grouped the different handwriting styles of digit 7 present in the test set. The elbow method and silhouette analysis were used jointly to determine the optimal number of clusters, identifying $k = 9$. The resulting cluster centres showed clear, interpretable differences in how digit 7 was written, and cluster sizes were relatively balanced across the nine groups. Overall, K-Means effectively partitioned the handwritten digit-7 samples into meaningful, interpretable clusters without using any label information.

7 Conclusion

This study examined two complementary problems on a reduced, binary subset of MNIST restricted to digits 1 and 7. For supervised classification, exploratory analysis, including PCA visualisation and pixel-level statistics, indicated that the two classes are close to linearly separable. This motivated a comparison between a linear model (Logistic Regression) and a non-linear kernel model (RBF SVM). After stratified five-fold cross-validation and grid-search tuning, Logistic Regression ($C = 0.1$, L2 penalty) achieved a mean cross-validation F1-score of 1.000, slightly outperforming the tuned RBF SVM ($C = 10$, $\gamma = 0.001$) at 0.970. This supports the conclusion that, for this near-linearly-separable binary problem, the additional representational capacity of the RBF kernel is unnecessary and may make cross-validated generalisation marginally less stable than the simpler linear decision boundary.

For unsupervised analysis, K-Means clustering applied to the 500 digit-7 test images, guided

jointly by the elbow method and silhouette analysis, identified $k = 9$ as a suitable number of clusters. The resulting cluster centres corresponded to interpretable handwriting-style variations, namely differences in stroke angle, size, and the shape of the horizontal crossbar. Cluster sizes were reasonably balanced, indicating that K-Means recovered meaningful structure without using any label information.

Two limitations of the present study should inform future work. First, both classifiers were evaluated only via training-set and cross-validation metrics; the 1,000-image held-out test set described in Table 1 was not used for a final, independent evaluation, and doing so would provide a stronger check against overfitting than cross-validation alone. Second, the classification task was restricted to two visually distinct digits and a small training sample (100 images); the results may not generalise to the full ten-class MNIST problem or to digit pairs that are less visually separable. Future work could extend this analysis to the complete digit set, incorporate a confirmatory test-set evaluation, and examine whether the handwriting-style clusters identified for digit 7 correspond to systematic differences in classifier error rates.

References

- Byun, H., Kim, M., Park, J. and Lee, S. (2024) ‘Research on image classification of RBF kernel SVM’, *IEEE International Conference on Intelligent Computing and Signal Processing*. IEEE. <https://doi.org/10.1109/ICSP60620.2024.10948583>.
- Géron, A. (2022) *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. 3rd edn. Sebastopol, CA: O’Reilly Media.
- Govender, P. and Sivakumar, V. (2020) ‘Application of k-means and hierarchical clustering techniques for analysis of air pollution: A review (1980–2019)’, *Atmospheric Pollution Research*, 11(1), pp. 40–56. <https://doi.org/10.1016/j.apr.2019.09.009>.
- Jolliffe, I.T. and Cadima, J. (2016) ‘Principal component analysis: A review and recent developments’, *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065), p. 20150202. <https://doi.org/10.1098/rsta.2015.0202>.
- Karuppasamy, A., Abdessalam, A., Hedjam, R., Zidoum, H. and Al-Bahri, M. (2024) ‘Feed-forward networks using logistic regression and support vector machine for whole-slide breast cancer histopathology image classification’, *Intelligence-Based Medicine*, 9, p. 100126. <https://doi.org/10.1016/j.ibmed.2023.100126>.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998) ‘Gradient-based learning applied to document recognition’, *Proceedings of the IEEE*, 86(11), pp. 2278–2324. <https://doi.org/10.1109/5.726791>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) ‘Scikit-learn: Machine learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Vasan, D. and Surendiran, B. (2024) ‘Comparative analysis of cross-validation techniques: LOOCV, k-folds cross-validation, and repeated k-folds cross-validation in machine learning models’, *American Journal of Theoretical and Applied Statistics*, 13(5), pp. 127–137. <https://doi.org/10.11648/j.ajtas.20241305.13>.